

SecureMail Shield

Guía completa del proyecto

Servicios externos, arquitectura, flujo del sistema, cómo arrancar el proyecto.

1. Servicios externos que usa tu proyecto

Tu proyecto se apoya en **5 servicios externos** porque cada uno hace algo que tu Django solo no puede:

Servicio	Para qué sirve	Cómo se usa
PostgreSQL	Guardar todos los datos (usuarios, alias, correos recibidos, reportes de sandbox).	base de datos, Puerto 5432.
Docker Desktop	Ejecutar el sandbox aislado donde se analiza el malware.	App de escritorio que arrancas tú.
ngrok	Dar a tu Django local una URL pública en internet.	Get que tunela localhost:8000 → URL pública.
SendGrid	RECIBIR los correos que llegan a tus alias @SecureMail.Shield.	Servicio en la nube que recibe SMTP y te lo manda como webhooks.
Resend	ENVIAR las alertas de amenaza desde alerts@PocketShield al que tú configuras.	API que te permite enviar emails desde Django.
Gmail SMTP	ENVIAR los correos de "recuperar contraseña" Django se conecta a smtp.gmail.com desde tu cuenta personal.	
Groq	Analizar los reportes del sandbox con IA (Llama 3.1 70B).	API que recibe un prompt y devuelve veredicto en lenguaje humano.

¿Por qué SendGrid Y Resend? ¿No es lo mismo?

No, son lo opuesto:

- **SendGrid** → **recibir** correos entrantes (los que llegan al alias). Tiene "Inbound Parse" que recibe los correos y los reenvía a tu webhook.
- **Resend** → **enviar** correos salientes (las alertas que mandas a tu Gmail real cuando detectas malware).

Podrías usar uno solo si quisieras (Mailgun, por ejemplo, hace ambas), pero como ya tenías Resend configurado para enviar antes de tener el dominio, lo dejaste así. Es perfectamente válido.

¿Por qué Groq y no usas tu propia detección?

Los analyzers (YARA, oletools, sandbox) detectan el **comportamiento técnico** del archivo (matches de reglas, hashes, patrones). Pero el reporte es **demasiado técnico** para un usuario normal — ve "YARA: Suspicious_Powershell_Loader" y no entiende.

Groq con Llama traduce todo eso a lenguaje natural: *"Este archivo es un descargador de PowerShell que intenta bajar otro malware. Recomendamos eliminar el alias y reportar al remitente."*

El veredicto técnico lo da YARA, la **explicación amigable** la da Groq.

2. Carpetas y archivos del proyecto

app/services/

Lógica de negocio separada de las vistas (patrón MVC). Cada archivo agrupa una responsabilidad:

- **alias_service.py** → genera direcciones de alias (amazon_x7k2@dockershield.lat).
- **auth_service.py** → autenticación + rate limiting de login (bloquea fuerza bruta).
- **email_service.py** → wrapper para enviar correos por Gmail SMTP.
- **password_reset_service.py** → genera tokens de recuperación + envía el correo.
- **profile_service.py** → manejo del avatar del usuario.
- **stats_service.py** → cálculos del dashboard (correos hoy, esta semana, etc.).

Por qué existe: si pones todo en las views, el código se vuelve un infierno. Las views solo coordinan; los services hacen el trabajo pesado.

app/sandbox/

El motor de análisis de malware. Tres niveles:

- **service.py** → recibe un archivo y dispara el contenedor Docker (ejecuta `docker run --network none ...`).
- **run_analysis.py** → script que CORRE DENTRO del contenedor, decide qué analyzer usar según el tipo de archivo.
- **analyzers/** → 9 archivos especializados:
 - yara_analyzer.py** → corre reglas YARA contra el binario.
 - executable_analyzer.py** → analiza .exe/.dll (PE headers, packers).
 - office_analyzer.py** → docs Office con macros VBA.
 - pdf_analyzer.py** → PDFs con JavaScript embebido.
 - archive_analyzer.py** → ZIP/RAR/7Z (extrae y analiza recursivo).
 - script_analyzer.py** → scripts (.bat, .ps1, .vbs, .sh) buscando patrones.
 - dynamic_executor.py** → ejecuta el script bajo strace para ver qué llama.
 - url_analyzer.py** → analiza URLs sospechosas.
 - body_analyzer.py** → cuerpo del correo (phishing, brand impersonation).

Dockerfile.sandbox

La "receta" de cómo construir el contenedor del sandbox. Tiene Python 3.12 + las librerías para analizar (yara-python, oletools, pefile, etc.). Cada vez que cambias algo del sandbox, lo rebuilds con:

```
docker build -f Dockerfile.sandbox -t email_seguro_sandbox .
```

docker/, docker-compose.yml y nginx.conf

Configuración para deployar TODO con docker-compose (Django + Postgres + sandbox + nginx) en un servidor de producción. **No los usas en desarrollo** — son para cuando un día montes esto en un VPS y quieras tener nginx como reverse proxy delante de Django. Por ahora puedes ignorarlos.

test_samples/

Carpeta para tus muestras de prueba del sandbox:

- **eicar.txt** → archivo de prueba estándar de antivirus.
- **suspicious.bat** → script con patrones maliciosos (no es real malware).
- **examen_final.pdf** → un PDF para probar.
- **run_tests.py** → suite de tests automatizados del sandbox.

*Esta carpeta está en la **exclusión de Defender** que añadimos antes, para que no borre los samples.*

.env

TODOS los secretos: SECRET_KEY de Django, password de Postgres, API keys de Resend/Groq, contraseña de aplicación de Gmail, etc. **Nunca subir a git** (el .gitignore ya lo excluye).

media/

Donde Django guarda los archivos subidos (avatares de usuarios + adjuntos de correos recibidos). También en exclusión de Defender.

3. Docker Desktop: cómo saberlo y por qué

Por qué hay que tenerlo abierto

Tu sandbox **NO** es código Python que corre en Django. Es un **proceso aparte** dentro de un contenedor Linux completamente aislado. Cuando llega un correo con adjunto, tu webhook ejecuta literalmente esto:

```
docker run --rm --network none --read-only --memory 256m \
-v ARCHIVO:/tmp/sample:ro email_seguro_sandbox \
python /app/sandbox/run_analysis.py /tmp/sample
```

Si Docker Desktop está cerrado, el comando docker no funciona → el sandbox falla → el correo se guarda pero sin análisis.

Cómo saber si está corriendo

Visualmente: el icono de la ballena de Docker en la barra de tareas (esquina inferior derecha). Si la ballena está blanca/quieta = corriendo. Si está ausente o "loading" = apagado/iniciando.

En terminal:

```
docker info 2>&1 | head -5
```

Si responde con info del servidor → activo. Si dice Cannot connect to the Docker daemon → cerrado.

Cómo saber si tu imagen del sandbox existe

```
docker images email_seguro_sandbox
```

Si aparece la línea, tienes la imagen lista. Si no aparece, hay que rebuildar:

```
docker build -f Dockerfile.sandbox -t email_seguro_sandbox .
```

4. ¿Por qué ngrok antes que runserver (o después)?

En realidad **el orden no importa**, pero es más cómodo así:

1. Docker Desktop primero (tarda 30s en arrancar, mejor adelantarlo).
2. Django runserver segundo (rápido).
3. ngrok tercero — y verás en su output que se conecta a localhost:8000. Si ngrok arranca antes que Django, también funciona, pero hasta que Django no esté arriba ngrok te dará error 502 si alguien hace request.

Lo importante: si reinicias Django, NO necesitas reiniciar ngrok. ngrok solo tunela; no le importa si Django se cae y vuelve. Pero si reinicias ngrok, sí debes verificar que la URL siga siendo la misma (con --url=twilight-baking-viewing.ngrok-free.dev siempre será la misma).

5. El flujo completo paso a paso

Con todo lo de arriba, este es el viaje de un correo:

- [1] Spammer manda correo: "spam@phisher.com" envia a "amazon_x7k2@dockershield.lat"
- |
- [2] DNS de Internet busca el MX de dockershield.lat -> apunta a SendGrid
- |
- [3] SendGrid recibe el correo SMTP, lo parsea, hace POST con multipart/form-data
- |
- [4] El POST viaja a https://twilight-baking-viewing.ngrok-free.dev/webhook/inbound/
- |
- [5] ngrok recibe ese POST publico y lo tunela a tu localhost:8000
- |
- [6] Django webhook (app/webhook.py) recibe el request, parsea campos
- |
- [7] Busca el alias en Postgres -> encuentra al usuario real (jgomez10@unemi.edu.ec)
- |
- [8] Crea EmailMessage en BD -> aparece en /bandeja/ casi inmediato
- |
- [9] Para cada adjunto:
Django llama a docker run -> arranca contenedor email_seguro_sandbox
El contenedor corre run_analysis.py -> ejecuta YARA + analyzers segun tipo
Devuelve JSON con score, evidencia, IOCs
- |
- [10] Combina todos los reportes (cuerpo + adjuntos + URLs)
- |
- [11] Si score >= 61: Django llama a Resend API -> manda alerta HTML
Resend usa tu DKIM/SPF de dockershield.lat -> llega a la inbox real
- |
- [12] Cuando el usuario abre el reporte en /sandbox/reporte/<id>/:
El frontend hace POST a /ai-analysis/ con todos los datos del reporte
Django llama a Groq API -> Llama 3.3 traduce el reporte a español sencillo
Aparece el veredicto IA en la pagina

6. Checklist rápido para arrancar

- # 1. Abrir Docker Desktop (icono barra de tareas, esperar ballena estable)
- # 2. Activar venv
ent_email\Scripts\activate
- # 3. Terminal A: Django
python manage.py runserver
- # 4. Terminal B: ngrok con dominio fijo
ngrok http --url=twilight-baking-viewing.ngrok-free.dev 8000
- # 5. (Opcional) Verificar que todo esta vivo:
docker info | head -3
curl http://localhost:8000/ # Django
curl https://twilight-baking-viewing.ngrok-free.dev/ # ngrok -> Django

Si los 4 puntos están OK, tu proyecto recibe correos reales y los analiza al instante.